The ABBA Numerical-Method Family

Duplicated phase space, projection, composition, and configuration axes

1 Purpose and model boundary

The ABBA model is a family of integrators built from the same explicit A–B–B–A map on two copies of a physical state. A public class selects the order and projection recipe; configuration controls select the projection equations, nonlinear solver, state extension, and optional physical energy tracking. These choices are configurations of a model, not additional model identities.

The family has four public method classes:

- `ABBA2Midpoint` and `ABBA2Implicit`;
- `ABBA4Implicit`, with one outer projection; and
- `ABBA6Implicit`.

Their focused theory documents live in the sibling model directories.

2 Duplicated-space ABBA map

Let $\dot{z} = f(t, z)$ with $z \in \mathbb{R}^m$. Duplicate the state as (u, v) and set $s = h/2$. One signed endpoint-time step beginning at t is

$$u_1 = u_0 + sf(t, v_0), \qquad v_1 = v_0 + sf(t, u_1), \tag{1}$$

$$v_f = v_1 + sf(t + h, u_1), \qquad u_f = u_1 + sf(t + h, v_f). \tag{2}$$

We denote this map by $\Psi_{h,t}(u_0, v_0) = (u_f, v_f)$. Its reverse is $\Psi_{-h,t+h}$, so the endpoint-time convention is essential for non-autonomous reversibility. When the split system is Hamiltonian, each line is a shear and the composition is symplectic on the duplicated space.

With

$$E = \begin{pmatrix} I \\ I \end{pmatrix}, \qquad P = \tfrac{1}{2}\begin{pmatrix} I & I \end{pmatrix}, \qquad G = \begin{pmatrix} I & -I \end{pmatrix}, \qquad N = G^T,$$

Ez embeds the physical diagonal, P takes its arithmetic mean, and $G(u, v) = u - v$ measures copy separation.

3 Projection choices

The midpoint method accepts $P\Psi_{h,t}(Ez)$ directly. The implicit methods use Hairer's symmetric projection. In the reduced formulation a multiplier $\mu \in \mathbb{R}^m$ displaces the input and corrects the output:

$$\widehat{Y} = Ez + N\mu, \qquad M = \Psi_{h,t}(\widehat{Y}), \tag{3}$$

$$r(\mu) = GM + 2\mu = 0, \qquad z^+ = P(M + N\mu). \tag{4}$$

If $J_\Psi = D\Psi_{h,t}(\widehat{Y})$, Newton uses

$$D_\mu r = GJ_\Psi N + 2I.$$

The simultaneous formulation solves for the corrected doubled output Y and μ together:

$$F(Y, \mu) = \begin{pmatrix} Y - N\mu - \Psi_{h,t}(Ez + N\mu) \\ GY \end{pmatrix} = 0.$$

Eliminating Y gives the reduced equation. Thus both formulations define the same ideal projected map when they converge to the same root. Newton uses the exact residual Jacobian; good Broyden replaces repeated Jacobian assembly by rank-one secant updates without changing the defining residual.

4 Canonical configuration space

ABBA2, ABBA4 and ABBA6 select different unprojected compositions and share one outer projection. Energy tracking is optional and passive. Signed accepted shear-stage traces update the passive momentum independently of the spatial root solve. The common state conventions are specified below.

5 Order by symmetric composition

The unprojected ABBA2 map is a symmetric method of order two in doubled space. Higher order is obtained by palindromic signed compositions, followed by one symmetric projection around the complete result. ABBA4 uses the triple jump (γ, δ, γ) with

$$\gamma = \frac{1}{2 - \sqrt[3]{2}}, \qquad \delta = -\frac{\sqrt[3]{2}}{2 - \sqrt[3]{2}}, \quad 2\gamma + \delta = 1, \quad 2\gamma^3 + \delta^3 = 0.$$

ABBA6 uses seven Yoshida coefficients whose first, third, and fifth odd moments have the required cancellations. These moment checks alone are not a full sixth-order proof; the Yoshida solution also satisfies the mixed commutator condition. Negative coefficients are intrinsic to these real compositions, so the dynamics must be defined at all internal signed times.

6 Projection placement is part of the model

Method	ABBA maps	projection policy
ABBA2Midpoint	1	arithmetic mean
ABBA2Implicit	1	one symmetric projection
ABBA4Implicit	3	one outer projection around the composition
ABBA6Implicit	7	one outer projection around the composition

The former three-projection fourth-order implementation has been removed. ABBA4 and ABBA6 now both have one outer projection. The retired single-projection ABBA4 factory is no longer an importable interface.

7 Finite solves and geometric claims

Symmetry, formal order, and symplecticity concern the ideal map obtained from the exact projection root. A finite stopping tolerance and floating-point arithmetic perturb that map. Consequently, the implementation records iterations, residual evaluations, final residual norms, tolerances, and projection multipliers. Step observers expose the accepted map so measured Jacobian defects can be distinguished from ideal algebraic properties.

8 Implementation correspondence

The existing ABBA classes select their recipes in the `order*.py` modules under `src/methods/extended/`. Their common run operations live in `abba.py`; the spatial engine is shared with BM4.

The public names, effective numerical defaults and exports from `simulation` remain unchanged. Internal imports use the defining modules; old import adapters have been removed. The ordinary implicit path is `advance`, `solve_projection`, `compose`, then `direct_map` or `adjoint_map`. Arithmetic midpoint variants use one explicit average instead of a nonlinear projection. Adjoint stages evaluate at the start of their signed substep; direct stages evaluate at its end.

Newton and Broyden share the spatial equations and convergence services. Exact particle-block tangents, passive energy and optional observation use retained shear inputs; energy is evaluated only after accepting the spatial map, with the existing factor $1/2$ for physical κ. Time and momentum never enter the spatial root. The new internal family name does not enable the retired full time/momentum projection.

Current module responsibilities and diagrams are documented in `docs/models/extended/simulation/extended-simulation-architecture.md`.

9 Common integration lifecycle

The executor initializes a fresh numerical instance through `IntegrationMethod.new_run` and starts the shared integration driver. Implicit orders share a family base. The fixed controller supplies accepted intervals and independent shadow samples; the coordinator owns progress, collection and event dispatch. Only sampled states and accepted metric rows are retained by the collector.

10 Physical accepted states and passive energy tracking

For N particles with d physical components, every accepted snapshot contains $Z \in \mathbb{R}^{dN}$ and one scalar time t. An optional energy observer owns separate physical momentum $\kappa \in \mathbb{R}^N$. Thus GC accepted vectors have $2N$ components and FC vectors have $4N$, whether energy is tracked or not. A doubled method prepares temporary $(u, v) \in \mathbb{R}^{2dN}$ copies, allows them to separate, and projects to one physical result before acceptance. The representation stores only the physical size and never owns a clock, momentum, physical coordinate declaration, dynamics or solver.

The normalized passive balance is

$$\dot{\kappa} = -\partial_t H(t, Z), \qquad \kappa(t_0) = 0, \qquad \Delta K(t) = H(t, Z(t)) + \kappa(t) - H(t_0, Z_0).$$

GC split-map contributions are summed in temporary work and divided by two to obtain physical momentum. Tracking leaves physical stages, nonlinear stopping scales and adaptive acceptance unchanged. An autonomous Hamiltonian supplies an explicit zero momentum derivative.

Hairer's constraint is solely $u - v = 0$: the reduced multiplier has $2N$ entries and ABBA's simultaneous spatial solve has $6N$ unknowns. Neither t nor κ is projected. The former eight-component projection is retired. Dynamics declare component order and position/velocity roles through immutable structural data; initial conditions and results retain those data independently. Execution injects $f(t, Z)$ and optional physical derivatives into a fresh method run. Methods own physical stages and projection equations, without retaining the dynamics object. Nonlinear tolerances, iteration limits, differentiation and iteration callbacks belong to the configured algebraic solver. Generic doubled copy operations live in `src/formulations/state.py`; classical, HBVM and adaptive methods need no state formulation.

Methods optionally return retained stage quadrature or a dense interpolant. The energy observer integrates momentum independently, using Hamiltonian functions supplied by execution. Controllers keep only physical accepted states; compiled JAX loops carry the observer accumulator separately. Observers receive physical states. Energy histories in `States.diagnostics` retain their existing labels; `extended_time` has shape (N, n_{samples}) and is broadcast from scalar sample times. Fixed shadow samples leave accepted states and counters unchanged. Adaptive quadrature uses accepted dense output and physical-only error control.